

Java Backend Architecture

Interview Series

Chapter 18.3

Load Balancing Algorithms

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Java Backend Architecture Interview Series

Chapter 18.3 – Load Balancing Algorithms

Overview

Nginx upstream load balancing distributes requests across a pool of Java backend instances, enabling horizontal scaling and fault tolerance. The choice of algorithm affects latency distribution, session affinity, and resilience to backend heterogeneity. Understanding each algorithm's trade-offs is essential for designing high-performance Java microservice deployments.

Key Definitions

Term	Definition
round_robin	Default algorithm. Requests distributed sequentially across servers respecting weights.
least_conn	Routes to server with fewest active connections. Optimal for variable-duration requests.
ip_hash	Client IP consistently routes to same server (session affinity). Hash changes if server removed.
hash \$key	Routes based on arbitrary key hash. consistent parameter enables consistent hashing (ketama).
random two	Picks the better of 2 random servers (power of two choices). Near-optimal distribution.
weight=N	Multiplies a server's share of traffic. server app1 weight=3 gets 3x more requests.
backup	Server used only when all primary servers are down.
max_fails	Number of failed attempts within fail_timeout before server marked unavailable.
fail_timeout	Window for counting max_fails AND duration server stays down after being marked.
zone name size	Shared memory for upstream state across all workers. Required for accurate least_conn.

Diagram 1: Load Balancing Algorithm Comparison

Algorithm	Distribution	Best For
round_robin weight=3,1	A B C A B C A B C AAA B AAA B	Uniform request durations Heterogeneous server capacity
least_conn	--> server with fewest active connections	Variable-duration Java API calls Long-running DB/external calls
ip_hash	hash(client_ip) mod N same client -> same server	Session-based Java apps (prefer stateless JWT instead)
hash \$uri consistent	hash(request_uri) mod N minimal remapping on change	Cache locality, CDN pre-warming Consistent key routing

Diagram 2: Passive Health Check State Machine

```

Server: UP
|
| max_fails failures in fail_timeout window
v
Server: DOWN (unavailable for fail_timeout seconds)
|
| fail_timeout expires -> single probe request sent
| success -----> Server: UP
| failure -----> DOWN (reset timer)
v
Requests routed to remaining UP servers
(if all down: 502 returned to client)

```

Code Examples

Round Robin with Weights:

```
upstream java_api {
    server app1:8080 weight=3;    # 60% of traffic
    server app2:8080 weight=2;    # 40% of traffic
    keepalive 32;
}
```

Least Connections (recommended for APIs):

```
upstream java_api {
    least_conn;
    zone backend 64k;            # required for cross-worker accuracy
    server app1:8080;
    server app2:8080;
    server app3:8080;
    keepalive 32;
}
```

Consistent Hash (URI affinity):

```
upstream java_api {
    hash $request_uri consistent; # same URI always goes to same server
    server app1:8080;             # consistent = ketama, minimal remapping
    server app2:8080;
    server app3:8080;
}
```

Health Check Tuning:

```
upstream java_api {
    least_conn;
    server app1:8080 max_fails=3 fail_timeout=30s;
    server app2:8080 max_fails=3 fail_timeout=30s;
    server backup1:8080 backup;
}
```

Interview Q&A – Load Balancing Algorithms

Q1. Why is `least_conn` better than `round_robin` for Java APIs?

Round robin ignores active connection counts. A slow Java endpoint (DB query, external call) causes one server to backlog while others are idle. `least_conn` routes to the server with fewest in-flight connections, naturally distributing around slow requests. For Spring Boot APIs with mixed fast (10ms) and slow (1s+) endpoints, `least_conn` reduces tail latency.

Q2. What is the power of two choices algorithm?

random two: Nginx picks 2 random upstream servers and routes to the one with fewer active connections. Achieves near-optimal distribution with less coordination overhead than global `least_conn`. Better than pure random (avoids hot spots). Useful in large upstream pools where `least_conn` lock contention is measurable.

Q3. What problem does consistent hashing solve?

Regular hash: when a server is added/removed, hash mod N remaps most keys -- cache miss storm for all Java cache instances. Consistent hashing (ketama): only K/N keys remap when 1 of N servers changes. Critical when Nginx routes same URI to same Java cache instance for in-memory caching or session affinity.

Q4. How does `ip_hash` handle server removal?

When a server is removed from `ip_hash` upstream, the hash space redistributes and many clients route to different servers -- session affinity broken. Use the 'down' keyword instead of removing: `server app2:8080 down;` -- preserves hash slots, excludes from routing. Only applicable when gracefully retiring a server. This is why stateless JWT auth is preferred over `ip_hash`-based sessions.

Q5. What happens when all upstream servers are marked down?

Nginx returns 502 Bad Gateway. If a backup server is configured, it activates when all primary servers are unavailable. Without backup: Nginx tries each server once and returns 502 after exhausting `proxy_next_upstream_tries`. Configure backup with a maintenance page server or a reduced-functionality Java endpoint for graceful degradation.

Q6. Why is the `zone` directive required for `least_conn`?

Without `zone`, each worker process maintains its own connection counts independently. With 4 workers, `least_conn` sees 1/4 of actual connections per server -- inaccurate. `zone` backend 64k allocates shared memory accessible by all workers, giving accurate global counts. Also required for Nginx Plus active health checks and the upstream API.

Q7. How do weights interact with `least_conn`?

With weights, the selection formula accounts for weight: $\text{effective load} = \text{active_connections} / \text{weight}$. Server with `weight=2` handles twice the connections before being considered 'more loaded' than `weight=1` server. Assign higher weights to larger Java instances (more CPU/memory) for proportional load distribution.

Q8. How does DNS-based upstream discovery work in Kubernetes?

Use `resolver` directive with variable in `proxy_pass`: `resolver kube-dns.kube-system.svc.cluster.local valid=10s;` set `$upstream` 'java-service.default.svc.cluster.local'; `proxy_pass http://$upstream`. Nginx re-resolves DNS at `valid=` interval, picking up new pod IPs. Without variable, hostname resolved once at startup -- misses scaling events.

Q9. How do you implement sticky sessions without `ip_hash`?

Cookie-based: Nginx Plus sticky cookie directive inserts a server-mapping cookie. Open-source: extract session cookie value and use `hash $cookie_JSESSIONID` consistent to route based on it. Best practice: eliminate stickiness entirely by using stateless JWT tokens -- session state in token, no server affinity required.

Q10. How do you drain a server for zero-downtime deployment?

Nginx Plus: `PUT /api/upstreams/backend/servers/0` with `drain:true` stops new requests while active ones complete. Open-source: in Kubernetes, fail the pod's readiness probe -- Nginx Ingress Controller removes the pod from the Endpoints object and stops routing to it. New deployment starts fresh pods. No requests dropped if readiness probe is failing before pod termination.